Bayesian Learning - Computer Lab 2

Liuxi Mei Xiaochen Liu

2025-04-28

1. Linear and polynomial regression

The dataset `temp_linkoping.csv` contains daily average temperatures (in degree Celsius) in Linköping between July 2023 and June 2024. Import the dataset in R.
The response variable is `temp` and the covariate `time`, which is defined as

$$\text{time} = \frac{\text{the number of days since the beginning of the observation period}}{365}.$$

A Bayesian analysis of the following quadratic regression model is to be performed:

$$\text{temp} = \beta_0 + \beta_1 \cdot \text{time} + \beta_2 \cdot \text{time}^2 + \varepsilon, \quad \varepsilon \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

(a) Use the conjugate prior for the linear regression model. The prior hyperparameters μ_0, Ω_0, ν_0 and σ_0^2 shall be set to sensible values. Start with

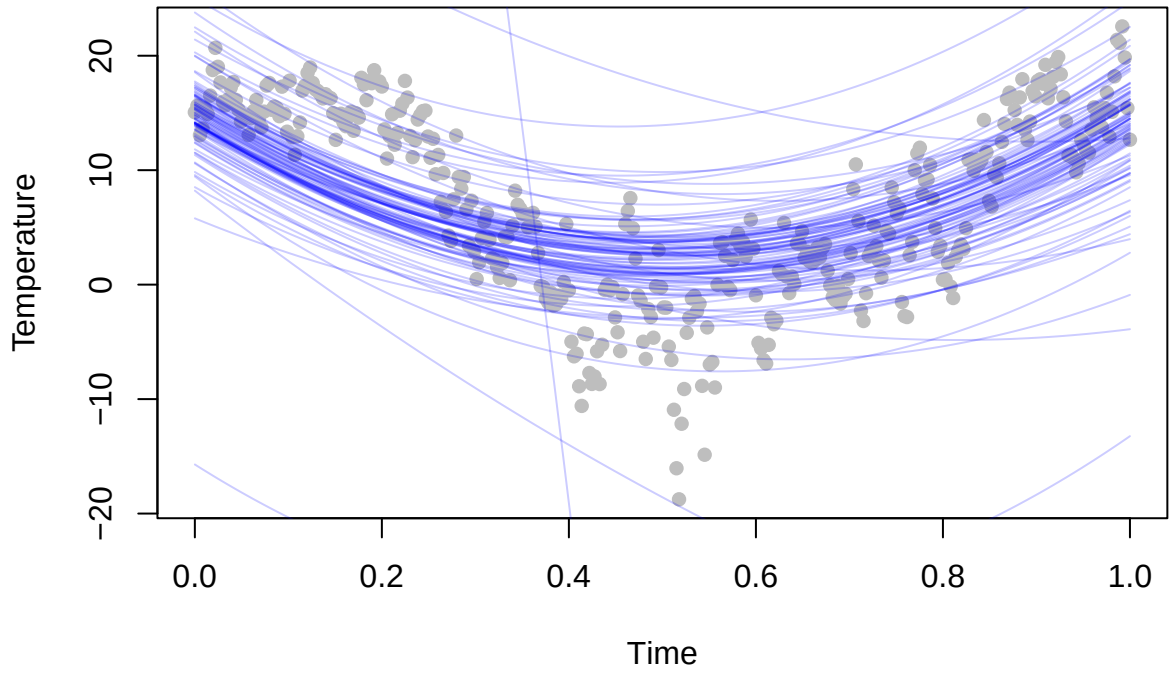
$$\mu_0 = (0, 100, -100)^T, \quad \Omega_0 = 0.01 \cdot I_3, \quad \nu_0 = 1, \quad \sigma_0^2 = 1.$$

Check if this prior agrees with your prior opinions by simulating draws from the joint prior of all parameters and for every draw compute the regression curve.
This gives a collection of regression curves; one for each draw from the prior.
Does the collection of curves look reasonable? If not, change the prior hyperparameters until the collection of prior regression curves agrees with your prior beliefs about the regression curve.
Hint: R package ***mvtnorm*** *can be used and your Scaled-Inv-*χ^2 simulator of random draws from Lab 1.

```
data <- read.csv('temp_linkoping.csv')
library (mvtnorm)

# define inverse chi square function
mu_0 <- c(15, -50, 50)
nu_0<- 1
sig_0_sq <- 2
omega_0 <- diag(3)*0.8
 rinvchisq <- function(n, df, scale) {
  df * scale / rchisq(n, df)
  }
```

Join prior for bete given

```r
# sort our time and time^2
data['time^2'] <- data[3]^2
data['time^0'] <- data[3]^0
time <- as.matrix(data[, c(6,3,5)])
temp <- as.matrix(data[, 4])

curve <-  matrix(NA, nrow = 366, ncol = 100)

for  (i in 1: 100) {
  sig_sq_sp<- rinvchisq(1,nu_0, sig_0_sq)
  samp <- rmvnorm(n=1, mean= mu_0, sigma = sig_sq_sp*solve(omega_0))
  pred  <- time %*% as.numeric(samp)
  curve [,i] <- pred
}

plot(time[,2], temp, pch = 16, col = 'gray', xlab = "Time", ylab = "Temperature")
for (i in 2:100) {
  lines(data[,3], curve[,i], col = rgb(0,0,1,alpha=0.2))
}
```



(b) Write a function that simulates draws from the joint posterior distribution of β_0, β_1, β_2 and σ^2.

i. Plot a histogram for each marginal posterior of the parameters.

ii. Make a scatter plot of the temperature data and overlay a curve for the posterior median of the regression function

$$f(\text{time}) = \mathbb{E}[\text{temp}|\text{time}] = \beta_0 + \beta_1 \cdot \text{time} + \beta_2 \cdot \text{time}^2,$$

i.e., the median of $f(\text{time})$ is computed for every value of `time`.
In addition, overlay curves for the 90% equal tail posterior probability intervals of $f(\text{time})$, i.e., the 5 and 95 posterior percentiles of $f(\text{time})$ are computed for every value of `time`.
Does the posterior probability interval contain most of the data points? Should they?

Part 1 Since we use a Normal-Inverse-Chi-Squared prior on (β, σ^2), where:

- $\beta \mid \sigma^2 \sim \mathcal{N}(\mu_0, \sigma^2\Omega_0^{-1})$
- $\sigma^2 \sim \text{Inv-}\chi^2(\nu_0, s_0)$

Given observed data (X, y), the joint posterior distribution of β and σ^2 is: - The marginal posterior of σ^2 is:

$$\sigma^2 \mid y \sim \text{Inv-}\chi^2(\nu_n, \sigma_n)$$

with:

$$\nu_n = \nu_0 + n$$

and

$$\nu_n\sigma_n^2 = \nu_0\sigma_0^2 + y^\top y + \mu_0^\top\Omega_0\mu_0 - \mu_n^\top\Omega_n\mu_n$$

where the updated parameters are:

$$\Omega_n = \Omega_0 + X^\top X$$

$$\mu_n = \Omega_n^{-1}(\Omega_0\mu_0 + X^\top y)$$

- The conditional posterior of β given σ^2 and data is:

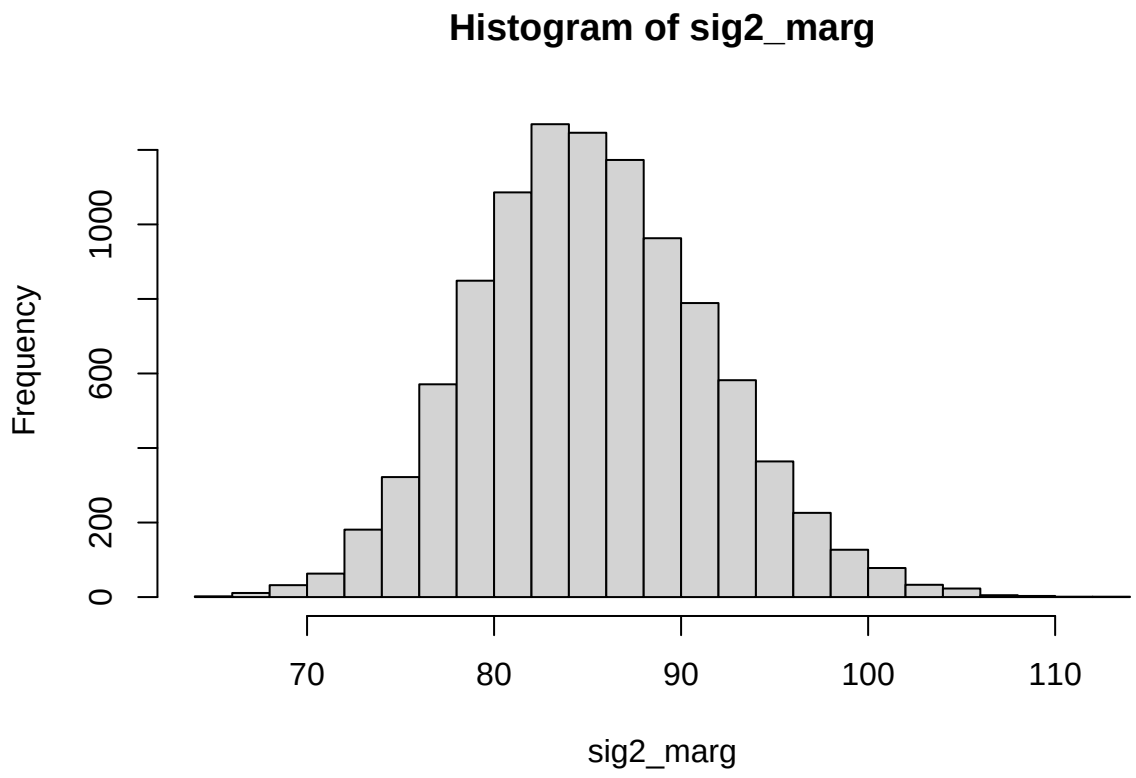
$$\beta \mid \sigma^2, y \sim \mathcal{N}(\mu_n, \sigma^2\Omega_n^{-1})$$

```
set.seed(123456)
sample_posterior <- function (number ,nu_0, sig_0_sq, omega_0) {
  # The marginal posterior of sigma

nu_n <- nu_0+366
omega_n <- t(time) %*%time+omega_0
mu_n <- solve(omega_n) %*% (omega_0%*%mu_0 +t(time)%*%(temp))
sig_n_sq <- as.numeric(1/nu_n*(nu_0*sig_0_sq+
                                 t(temp)%*%temp+
                                 t(mu_0)%*%omega_n%*%mu_0-
                                 t(mu_n)%*%omega_n%*%mu_n))

# sampled results
```

```r
# joint posterior of sigma square given data
  sig2_marg<- rinvchisq(number,nu_n, sig_n_sq)

# joint posterior of beta given sigma square and data
beta_marg <- matrix(nrow=number, ncol=3)
colnames(beta_marg) = c('beta0', 'beta1','beta2')

 for (i in 1: number) {
     beta_marg[i,] <- as.vector(rmvnorm(n=1, mean= mu_n, sigma = sig2_marg[i]*solve(omega_n)))
 }
return(list(sig2_marg = sig2_marg, beta_marg = beta_marg))
}

res <-sample_posterior(number=10000 ,nu_0=nu_0, sig_0_sq=sig_0_sq, omega_0=omega_0)
sig2_marg <- res$sig2_marg
beta_marg <- res$beta_marg
```

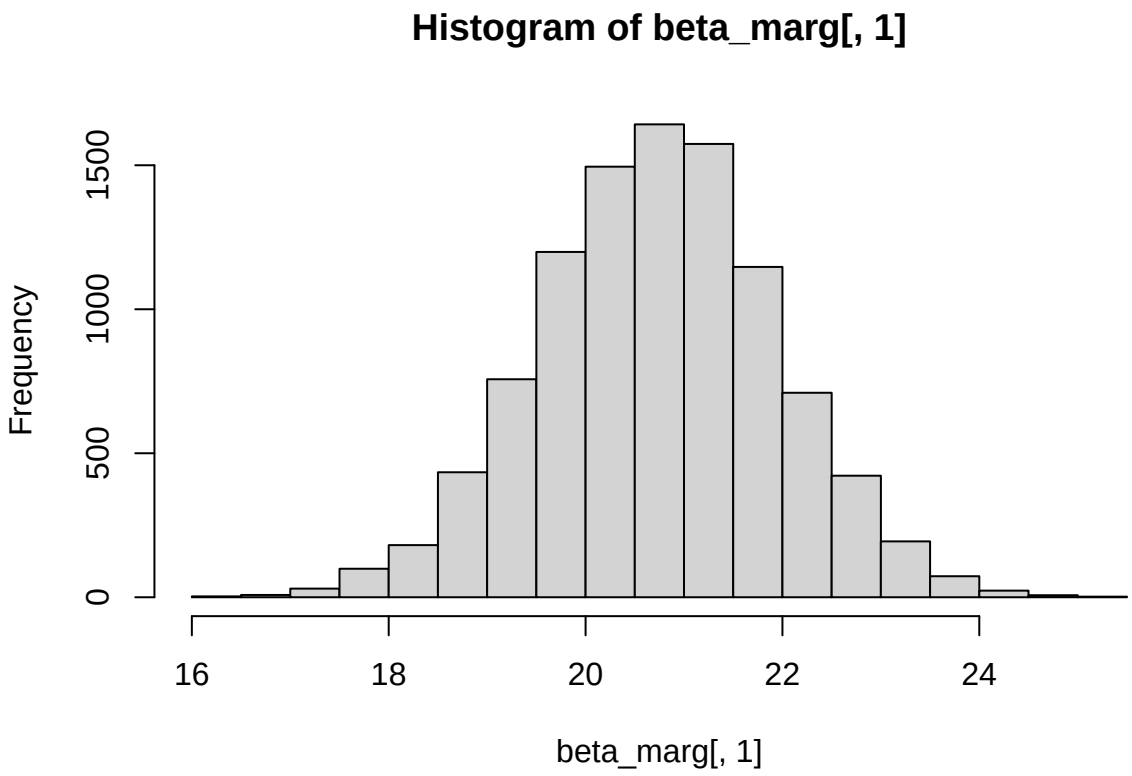
Then we can plot the histogram for σ^2 given data:

```r
hist(sig2_marg, breaks = 20)
```



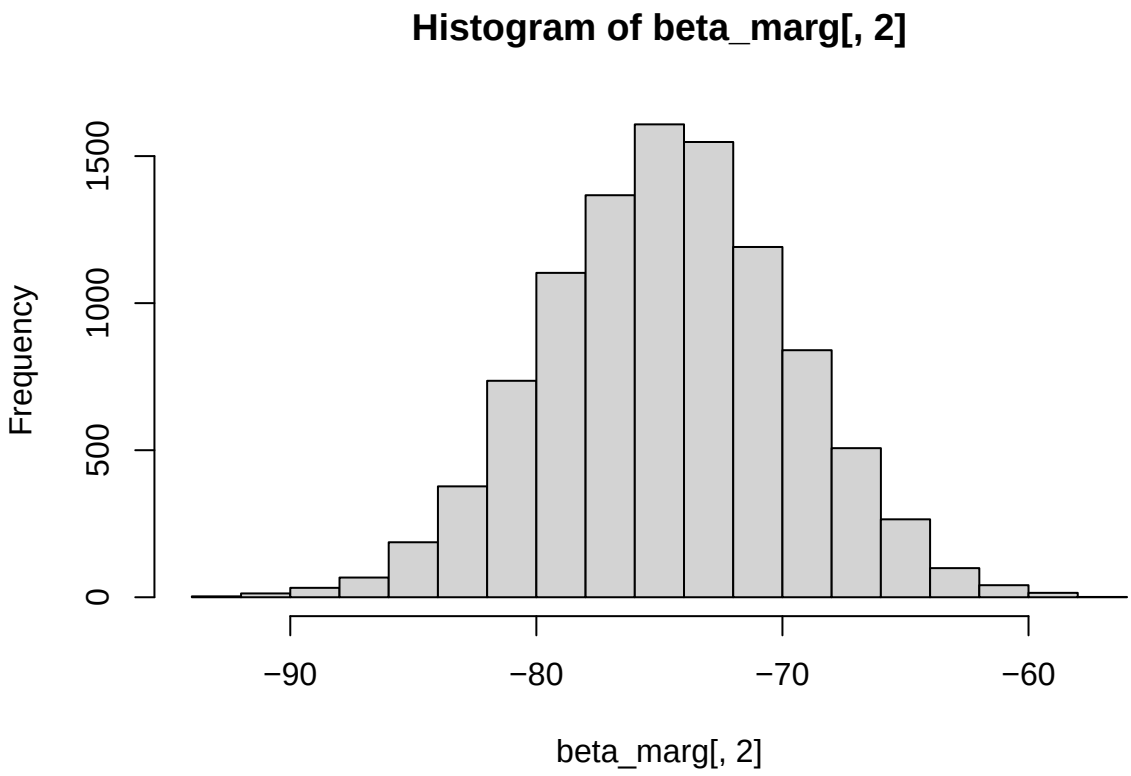
The histogram for β_0 given σ^2 and data:

```r
hist(beta_marg[,1], breaks = 20)
```



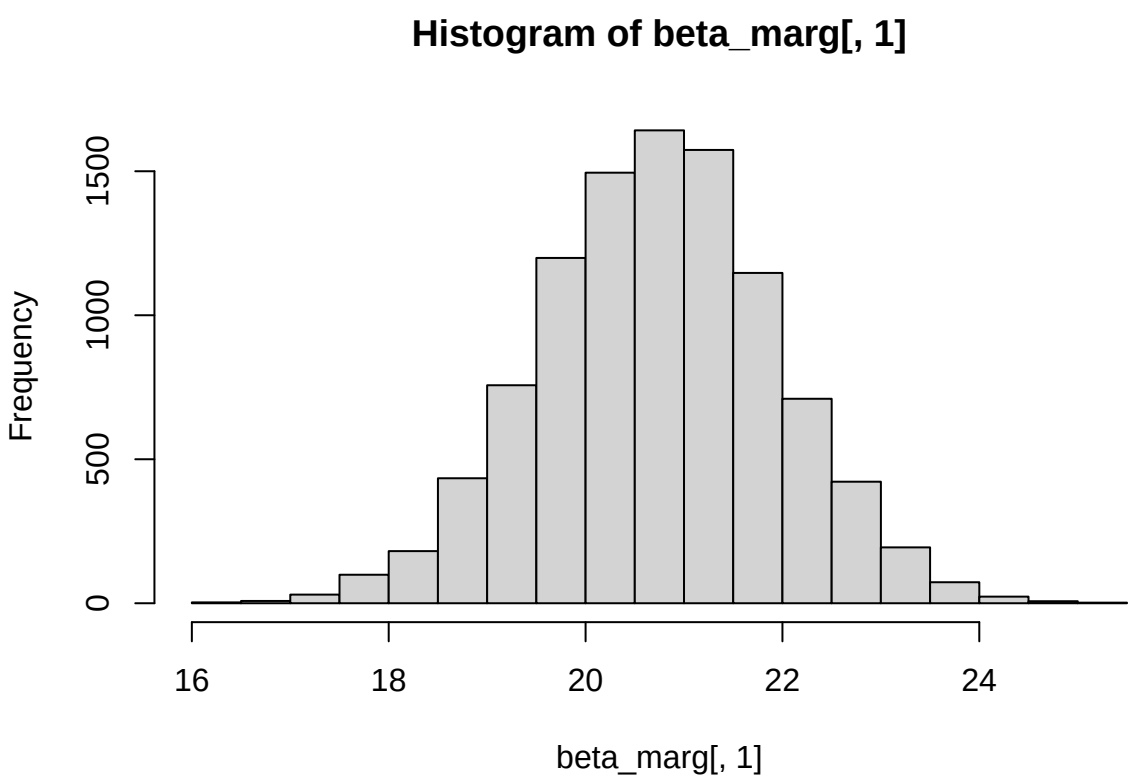
The histogram for β_1 given σ^2 and data:

```
hist(beta_marg[,2], breaks = 20)
```

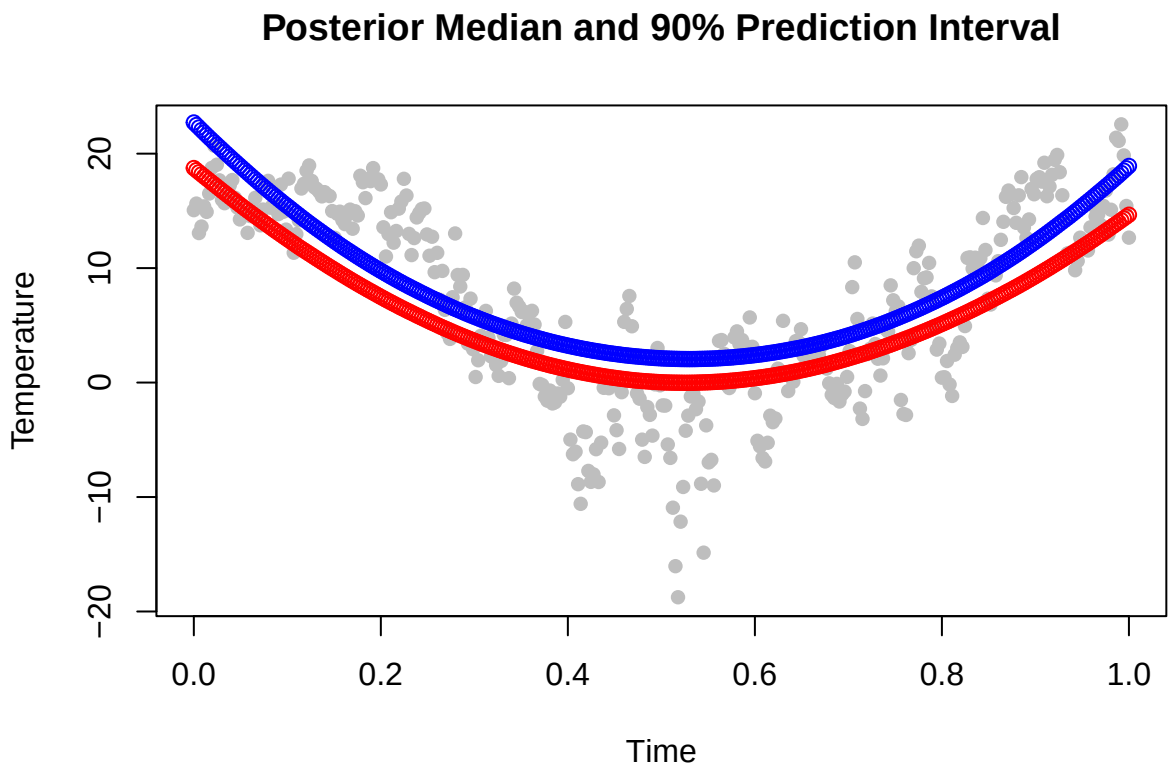


The histogram for β_2 given σ^2 and data:

```
hist(beta_marg[,1], breaks = 20)
```




```
pred_temp <- time%*%t(beta_marg)
pred_temp_meadian <- apply(pred_temp,1, quantile, probs = c(0.05,0.95) )

plot(time[,2], temp, pch = 16, col = 'gray', xlab = "Time", ylab = "Temperature",
     main = "Posterior Median and 90% Prediction Interval")
points(time[,2], pred_temp_meadian[1,], col ='red')
points(time[,2], pred_temp_meadian[2,], col ='blue')
```



Part 2

(c) It is of interest to locate the time with the lowest expected temperature (i.e., the time where $f(\text{time})$ is minimal). Let's call this value $\tilde{x}$.
Use the simulated draws in (b) to simulate from the posterior distribution of $\tilde{x}$.
You can solve this analytically or numerically.
Hint: The regression curve is a quadratic polynomial. Given each posterior draw of β_0, β_1, and β_2, you can find a simple formula for $\tilde{x}$.

To find the time point $\tilde{x}$ at which the function reaches its minimum , we compute the derivative with respect to time:

$$\frac{d}{d\,\text{time}} f(\text{time}) = \beta_1 + 2\beta_2 \cdot \text{time}$$

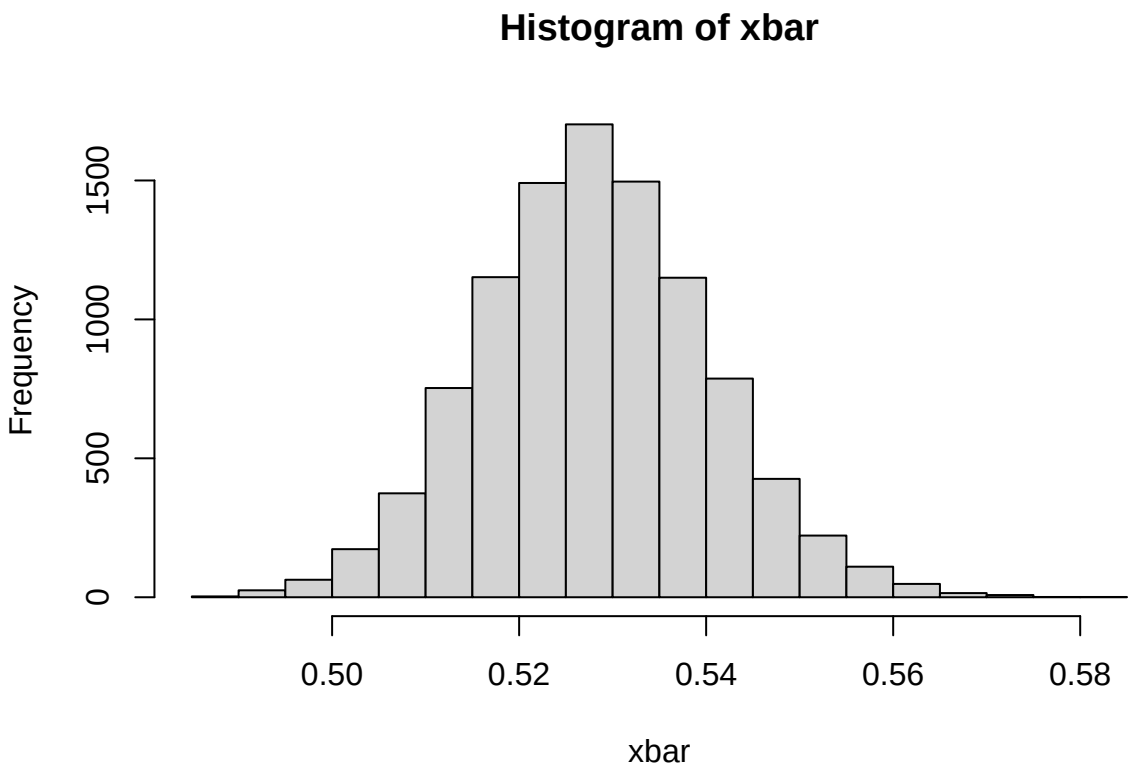
Setting the derivative equal to zero:

$$\beta_1 + 2\beta_2 \cdot \tilde{x} = 0$$

Solving for $\tilde{x}$ gives:

$$\tilde{x} = -\frac{\beta_1}{2\beta_2}$$

```
xbar <- -beta_marg[,2]/2/beta_marg[,3]
hist(xbar, breaks = 20)
```



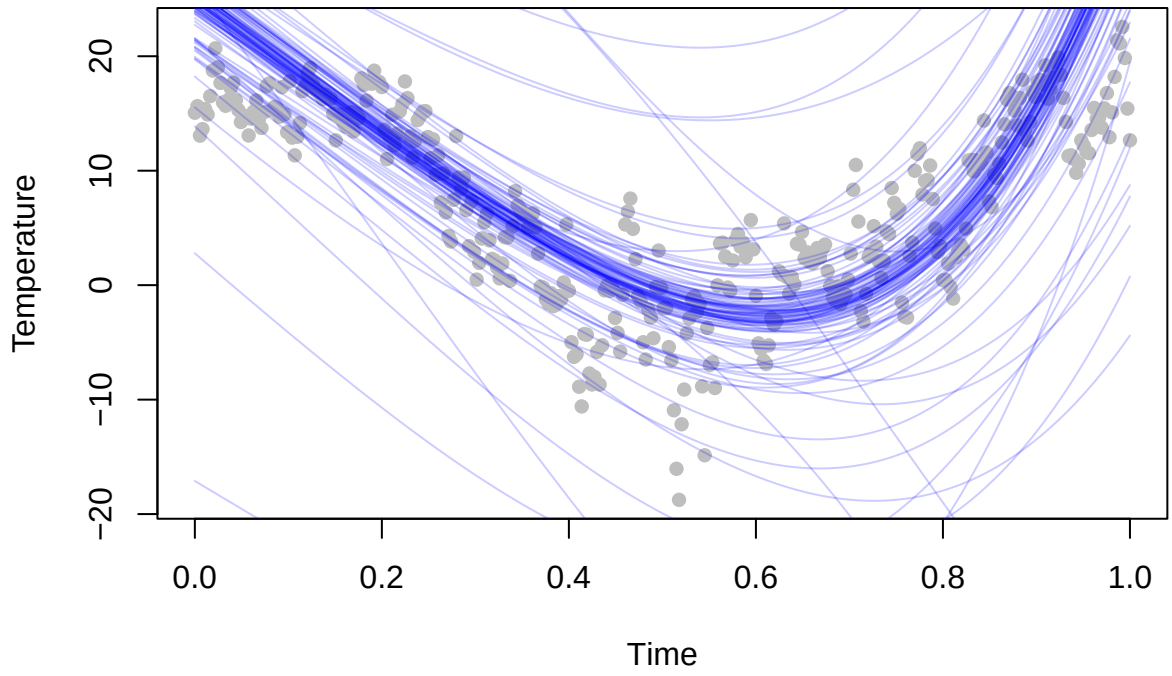
(d) Say now that you want to estimate a polynomial regression of order 10, but you suspect that higher order terms may not be needed, and you worry about overfitting the data.
Suggest a suitable prior that mitigates this potential problem and motivate your choice.
Repeat task (a) for the selected prior and the higher order polynomial to see if it behaves as expected.
Hint: The task is to specify μ_0 and Ω_0 in a suitable way.

To reflect our prior belief that lower-order terms are likely important, we assign relatively low precision values (small diagonal values (1) in Ω) to them.On the other hand, for higher-order terms (e.g., degree 5 to 10), we set much higher prior precision (1000), encouraging these coefficients to be small unless the data provides strong evidence otherwise.

It is shown that the curve fit the actual data better than what we have in task (a), but no significant improvement is obtained than what we have from task (b) where we have even lower order. Therefore, we believe that higher order terms are not needed.

```
# define inverse chi square function
mu_0_new <- c(25, -60, 0.01,-1,60,10,0,0,0,0,0)
nu_0<- 1
sig_0_sq <- 2
omega_0_new <- diag(c(rep(1, 6), rep(1000, 5)))

#construct poly terms
x <- data[, 3]
poly_terms <- sapply(0:10, function(p) x^p)
colnames(poly_terms) <- paste0("time^", 0:10)
time_new <- as.matrix(poly_terms)
```

```r
# sort our time and time^2

curve_new <-  matrix(NA, nrow = 366, ncol = 100)

for  (i in 1: 100) {
  sig_sq_sp<- rinvchisq(1,nu_0, sig_0_sq)
  samp_new <- rmvnorm(n=1, mean= mu_0_new, sigma = sig_sq_sp*solve(omega_0_new))
  pred_new  <- time_new %*% as.numeric(samp_new)
  curve_new [,i] <- pred_new
}

plot(time[,2], temp, pch = 16, col = 'gray', xlab = "Time",
     ylab = "Temperature"#,ylim=range(curve_new)
     )
for (i in 2:100) {
  lines(data[,3], curve_new[,i], col = rgb(0,0,1,alpha=0.2))
}
```



2. Posterior approximation for classification with logistic regression

The dataset Disease.xlsx contains n = 313 observations on the following seven variables related to a certain disease:

Variable	Data type	Meaning	Role
Class_of_diagnosis	Binary	Disease or not?	Response y
Gender	Binary	Woman (1) or Male (0)	Feature
Age	Counts	Age	Feature
Diagnosis_method	Binary	1 for a certain diagnosis method	Feature
Duration_of_symptoms	Numeric	Duration of symptoms in days	Feature
Dyspnoea	Binary	1 if person has laboured breathing	Feature
White_blood	Counts	N white blood cells per microliter	Feature

(a) Consider the logistic regression model:

$$\Pr(y = 1|x, \beta) = \frac{\exp(x^T\beta)}{1 + \exp(x^T\beta)},$$

where y equals 1 if the person has a disease and 0 if not. x is a 7-dimensional vector containing six features and a column of 1's to include an intercept in the model. The goal is to approximate the posterior distribution of the parameter vector with a multivariate normal distribution

$$\beta|y, x \sim \mathcal{N}(\tilde{\beta}, J_y^{-1}(\tilde{\beta})),$$

where $\tilde{\beta}$ is the posterior mode and $J(\tilde{\beta}) = -\frac{\partial^2 \ln p(\beta|y)}{\partial\beta\partial\beta^T}\Big|_{\beta=\tilde{\beta}}$ is the negative of the observed Hessian evaluated at the posterior mode. Note that $\frac{\partial^2 \ln p(\beta|y)}{\partial\beta\partial\beta^T}$ is a 7×7 matrix with second derivatives on the diagonal and cross-derivatives $\frac{\partial^2 \ln p(\beta|y)}{\partial\beta_i\partial\beta_j}$ on the off-diagonal. You can compute this derivative by hand, but we will let the computer do it numerically for you. Calculate both $\tilde{\beta}$ and $J(\tilde{\beta})$ by using the `optim` function in R.

Hint: You may use code snippets from my demo of logistic regression in Lecture 6. Use the prior $\beta \sim \mathcal{N}(0, \tau^2 I)$, where $\tau = 2$.
Present the numerical values of $\tilde{\beta}$ and $J_y^{-1}(\tilde{\beta})$ for the Disease data. Compute an approximate 95% equal tail posterior probability interval for the regression coefficient to the variable Age. Would you say that this feature is of importance for the probability that a person has the disease?

Hint: You can verify that your estimation results are reasonable by comparing the posterior means to the maximum likelihood estimates, given by:

```
library(mvtnorm)

data <- read.csv("Disease.csv")
Nobs <- dim(data)[1] # number of observations
#data <- data.frame(intercept=rep(1,Nobs),disease_data) # add intercept
data[,c("age","duration_of_symptoms","white_blood" )] <-
scale(data[,c("age","duration_of_symptoms","white_blood" )])

Xnames = colnames(data)[1:6]
LogPostLogistic <- function(beta,y,X,tau){
  lin_pred <- X %*% beta
  p = length(beta)
```

```
  log_lik <- sum(y * lin_pred - log1p(exp(lin_pred)))
  #log_prior <- -sum(beta^2) / (2 * tau^2)
  log_prior = dmvnorm(beta, rep(0, p), tau^2 * diag(p), log=TRUE);
  return(log_lik + log_prior)
}
lambda <- 1#
tau=2
Npar = dim(data)[2] - 1
mu <- as.matrix(rep(0,Npar)) #
initVal <- matrix(0,Npar,1)
Sigma <- (1/lambda)*diag(Npar)
y =  as.numeric(data$class_of_diagnosis)

X = as.matrix(data[,1:ncol(data) - 1])# Select which covariates/features to include
OptimRes <- optim(par = rep(0, ncol(X)),
                  fn = LogPostLogistic,
                  X = X, y = y, tau = tau,
                  method = "BFGS",
                  control = list(fnscale = -1, maxit = 1000),
                  hessian = TRUE)
approxPostMode <- matrix(OptimRes$par,1,6)
cat("The posterior mode:",OptimRes$par) # The posterior mode
```

```
## The posterior mode: -0.3882422 -0.2661479 -0.6423896 0.19313 -0.1375241 -0.04498936
```

```
cov_matrix <- solve(-OptimRes$hessian)  # J^{-1}
approxPostStd <- sqrt(diag(cov_matrix)) # `Computing approximate standard deviations.

Cred_int <- matrix(0,2,6) # Create 95 % approximate credibility intervals for each coefficient
Cred_int[1,] <- approxPostMode - 1.96*approxPostStd
Cred_int[2,] <- approxPostMode + 1.96*approxPostStd

colnames(Cred_int) <- Xnames
rownames(Cred_int) <- c("LCI","UCI") #LCI Lower Confidence Interval UCI Upper Confidence Interval
print(Cred_int)
```

```
##      Intercept         age     gender duration_of_symptoms   dyspnoea
## LCI -1.0010275 -0.51707150 -1.1350628          -0.04685009 -0.7496778
## UCI  0.2245431 -0.01522428 -0.1497163           0.43311005  0.4746297
##     white_blood
## LCI  -0.2954176
## UCI   0.2054389
```

```
#  glm
glm_data <- data.frame(
  Class_of_diagnosis = data$class_of_diagnosis,
  Gender = data$gender,
  Age_z = data$age,
  Duration_z = data$duration_of_symptoms,
  Dyspnoea = data$dyspnoea,
  White_z = data$white_blood
)
```

```r
glm_model <- glm(Class_of_diagnosis ~ Gender + Age_z + Duration_z + Dyspnoea + White_z,
                 data = glm_data, family = binomial)
print(summary(glm_model))
```

```
##
## Call:
## glm(formula = Class_of_diagnosis ~ Gender + Age_z + Duration_z +
##     Dyspnoea + White_z, family = binomial, data = glm_data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.38961    0.31999  -1.218   0.2234
## Gender      -0.64938    0.25415  -2.555   0.0106 *
## Age_z       -0.26762    0.12839  -2.084   0.0371 *
## Duration_z   0.19367    0.12270   1.578   0.1145
## Dyspnoea    -0.13348    0.31924  -0.418   0.6759
## White_z     -0.04504    0.12813  -0.352   0.7252
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 384.25  on 312  degrees of freedom
## Residual deviance: 369.86  on 307  degrees of freedom
## AIC: 381.86
##
## Number of Fisher Scoring iterations: 4
```

```r
cat("\n95% CI for Age coefficient: [", Cred_int[1,2], ", ", Cred_int[2,2], "]\n")
```

```
##
## 95% CI for Age coefficient: [ -0.5170715 ,  -0.01522428 ]
```

```r
cat("MAP of age coefficient: ", approxPostMode[2], "\n")
```

```
## MAP of age coefficient:  -0.2661479
```

```r
cat("Posterior mean of age coefficient: ", OptimRes$par[2], "\n")
```

```
## Posterior mean of age coefficient:  -0.2661479
```

```r
cat("glm model coefficient of age: ", coef(glm_model)[2], "\n")
```

```
## glm model coefficient of age:  -0.6493847
```

```r
#Report the posterior mean (Estimate) and 95% confidence interval (LCI, UCI),
 # Age:  -0.26762 (95% CI: -0.015, -0.51)
```

The 95% confidence interval of the Age variable does not include 0, result is [-0.015, -0.51] ,MAP is -0.26, so it has a significant impact on the probability of illness.

(b) Use your normal approximation to the posterior from (a). Write a function that simulate draws from the posterior predictive distribution of $\Pr(y = 1|x)$, where the certain diagnosis method was used and where the values of x corresponds to a 38-year-old woman, with 10 days of symptoms, no laboured breathing and 11000 white blood cells per microliter. Plot the posterior predictive distribution of $\Pr(y = 1|x)$ for this person.

Hints: The R package ***mvtnorm*** *will be useful. Remember that* $\Pr(y = 1|x)$ can be calculated for each posterior draw of .

```r
set.seed(123)
data_b <- read.csv("Disease.csv")

new_age <- (38 - mean(data_b$age)) / sd(data_b$age)
new_duration <- (10 - mean(data_b$duration_of_symptoms)) / sd(data_b$duration_of_symptoms)
new_white <- (11000 - mean(data_b$white_blood)) / sd(data_b$white_blood)
X_new <- c(1, 1, new_age, new_duration, 0, new_white)

samples <- rmvnorm(10000, mean = OptimRes$par, sigma = cov_matrix)
lin_pred <- samples %*% X_new
pr <- exp(lin_pred) / (1 + exp(lin_pred))

hist(pr, breaks = 50, main = "Posterior Predictive Distribution of Pr(y=1|x)",
     xlab = "Probability", col = "skyblue", border = "white")
```